inkledger

CASE STUDY

Your time, from every tracker, in order.

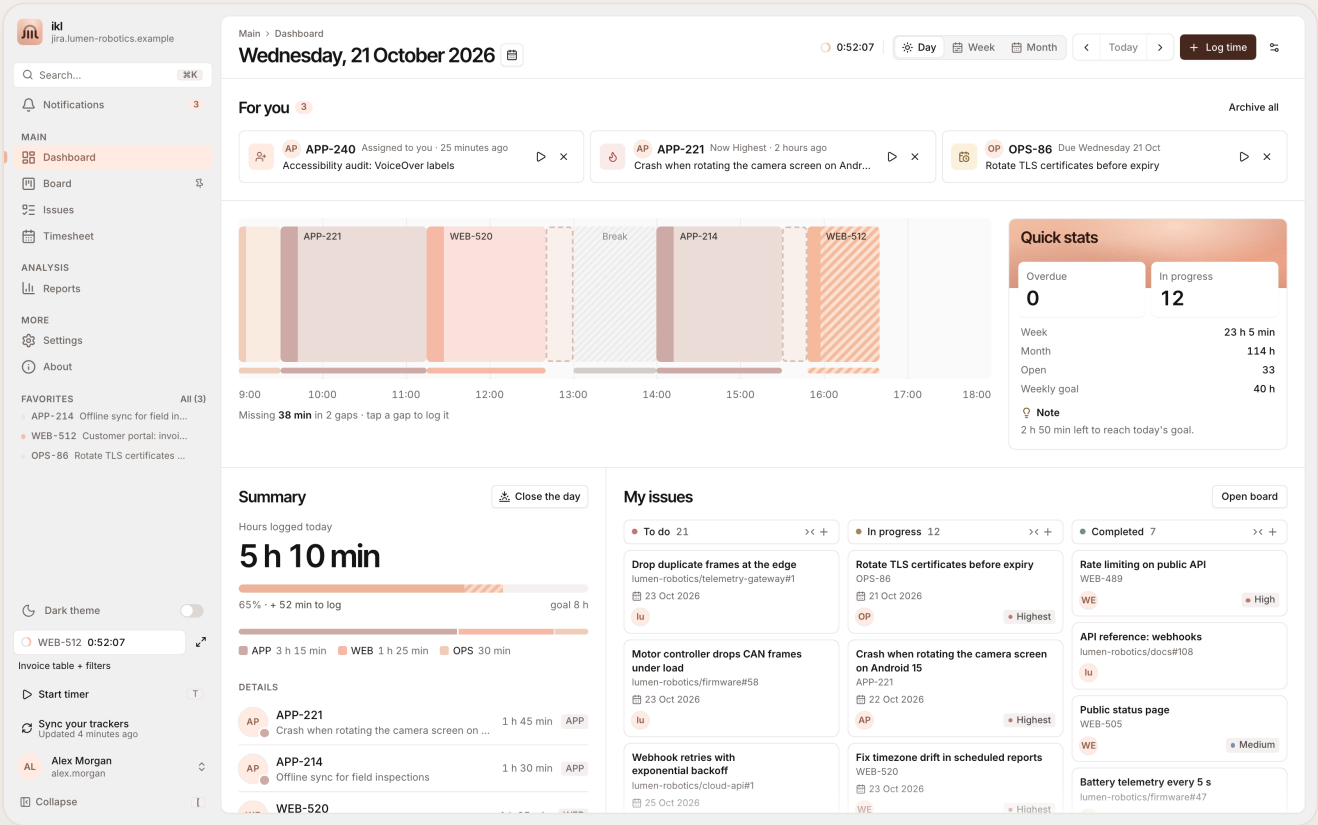
How a self-hosted, keyboard-first ledger gathers the issues of Jira, GitHub, GitLab and twenty more trackers in one place and turns logging time from a Friday chore into a few keystrokes a day. It is offline-safe, has native apps and is open source.

Product
Web app, native shell, PWA

Stack
React · tRPC · Hono · Postgres · Tauri

License
MIT

Date
October 2026



AT A GLANCE

One ledger for the whole loop: track, fill, log, review.

inkledger sits between you and the trackers you work in. It keeps your issues, from every tracker, in one fast local catalogue, runs your timers, shows which parts of the working day are still unlogged and writes real worklogs where a tracker keeps time. Nothing is written twice, and it keeps working when a tracker is unreachable.

3 months
of development, July to October 2026

68k lines
TypeScript and Rust across app, server, shared code and native shell

550+ tests
unit and integration tests plus 79 browser regressions

4 platforms
web, macOS and Windows (Tauri), Android, iPhone (PWA)

2 languages
English and Italian, app, server messages and native menus

0 telemetry
self-hosted; the only external services are your own trackers

Who it is for

People who work across more than one tracker and still have to account for their day: developers, consultants, small teams. Jira Cloud and **Server / Data Center** (localized servers included) and GitHub are built today. Each person connects their own accounts; no tracker is the base.

What makes it different

- Gaps of the working day are visible and fillable, not something you reconstruct at the end of the week.
- One provider contract for every tracker, so a new one is an adapter, not a rewrite.
- An offline queue that never duplicates a worklog, even with two servers on one database.
- Native apps and phones reach your own server over Tailscale or the local Wi-Fi.

Scope

Timers · Gap filling · Quick log · Dashboard · Issue catalogue · Jira + GitHub · Search language · Buckets · Kanban + transitions · Pinboard · Timesheet · Week locks · Reports + CSV · Notifications · Idle detection · Focus mode · Day review · Offline queue · Backups · Local accounts · Profiles · Remembered devices · Modules · i18n · Dark mode

THE PROBLEM

Time is easy to postpone and hard to reconstruct, in every tracker.

> “I worked a full day, but the timesheet says five hours. Where did the afternoon go?”

Friction in the trackers themselves

- Every worklog means opening the issue, opening a dialog and typing a duration and a start time.
- Nothing tells you which part of the day is still missing.
- Work is split across tools, and some of them keep no time at all.
- Context switches are frequent: a timer that follows you across issues, and across trackers, does not exist.

Constraints of the real setup

- A **Jira Server on premises**, localized in Italian: status categories and priorities have Italian names, so logic has to rely on stable keys.
- Corporate networks: a tracker is not always reachable (VPN drops, maintenance).
- A rejected credential on one tracker must not sign you out of everything.
- Credentials must stay on the user's machine.

Goals

Log in seconds.
Timers, quick presets, familiar duration syntax, keyboard everywhere.

See the holes.
The working schedule defines the day; what is not covered is shown and can be filled.

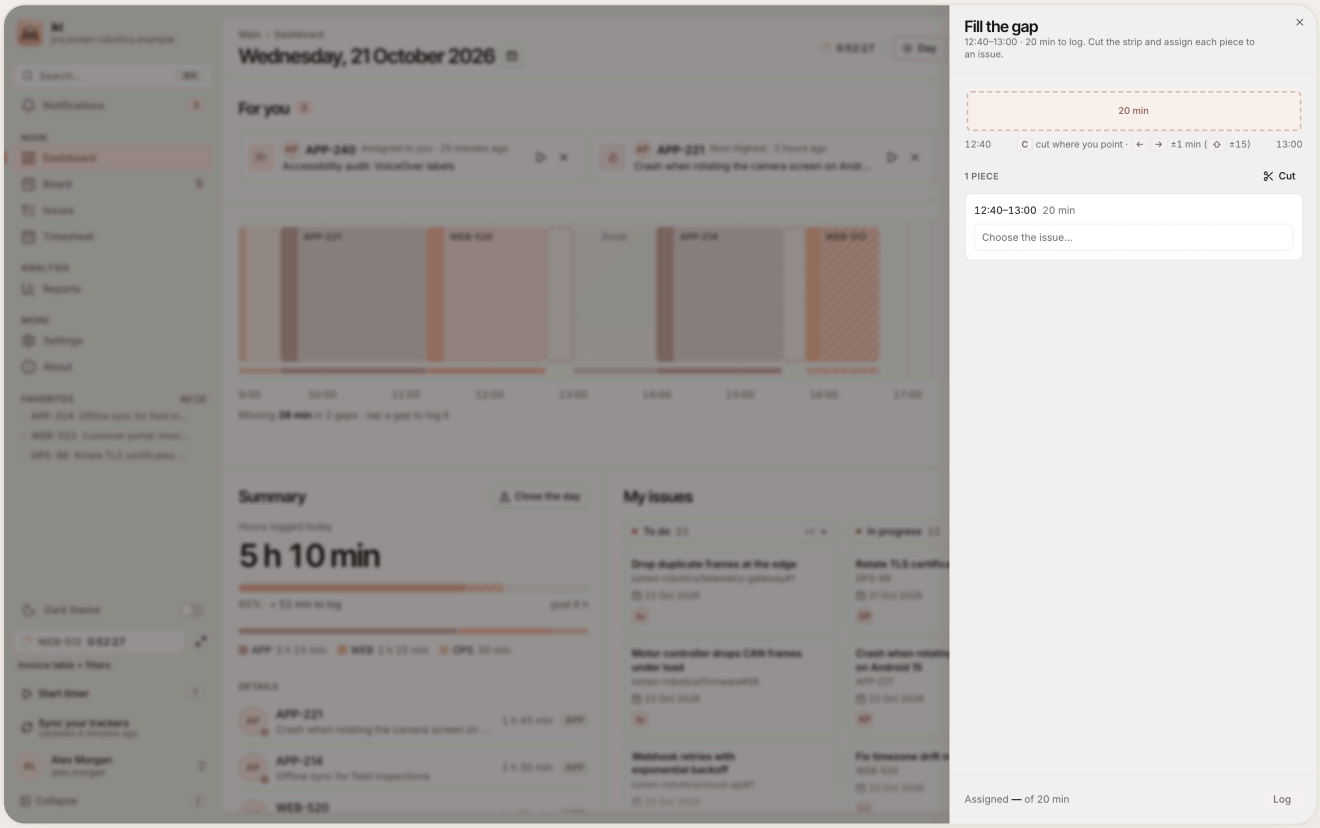
Never lose or double time.
An offline queue that is idempotent, safe concurrent servers, a trash for deleted worklogs.

Own the data.
Self-hosted, encrypted secrets, backups, no telemetry.

PRODUCT TOUR · TRACK

The day, as a timeline.

The dashboard draws today against the working schedule: logged blocks, breaks, the running timer (hatched) and the gaps (dashed). One tap on a gap opens **Fill the gap**: cut the strip with a click or the `C` key, nudge cuts by a minute with the arrows, assign each piece to an issue and log them all at once. Logged pieces are locked, so a span already logged can never move.



Fill the gap: a 20-minute hole, ready to be cut and assigned.

Timers, never lost

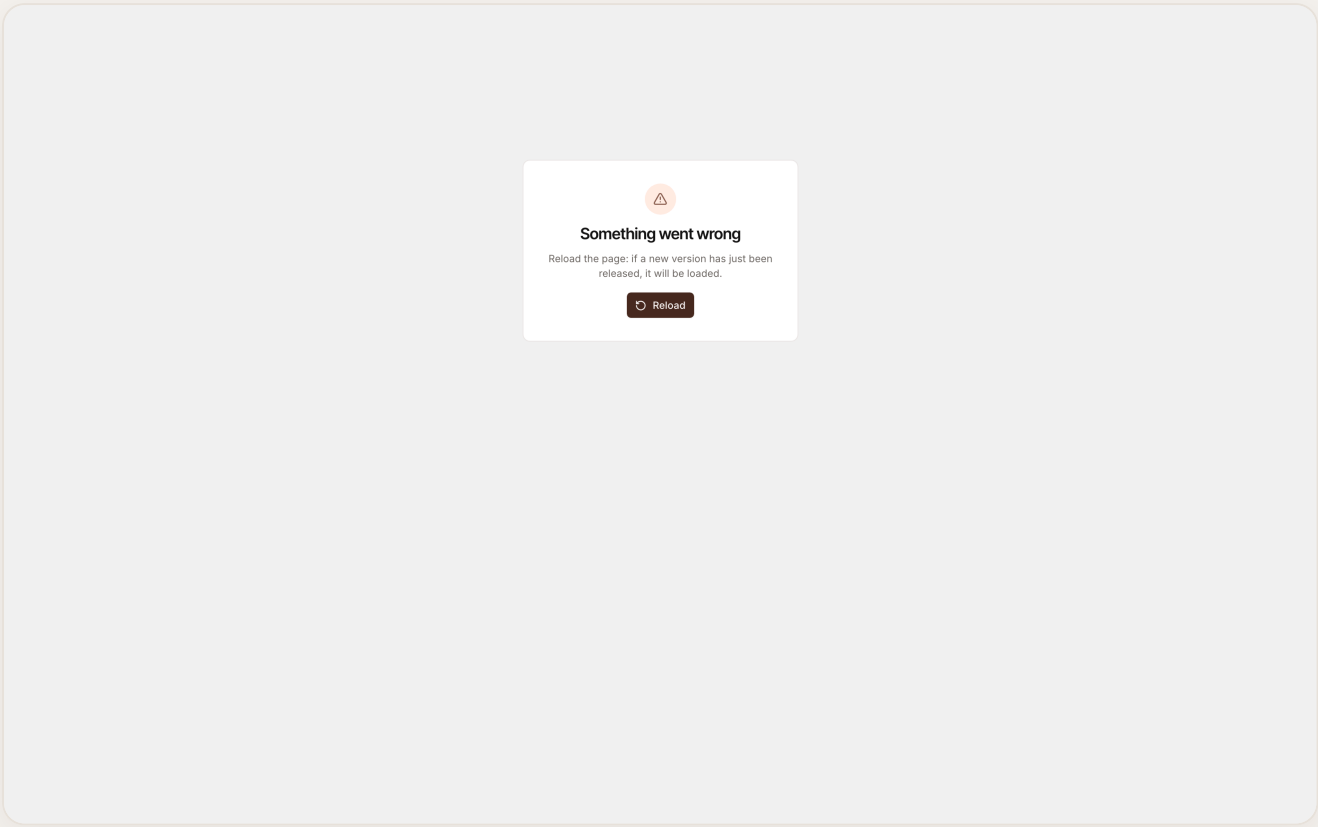
Starting a timer on another issue can stop the current one and queue "log this time"; parallel work is allowed too. Stopped timers survive until logged; the end-of-day review turns them into worklogs in one pass.

Away detection

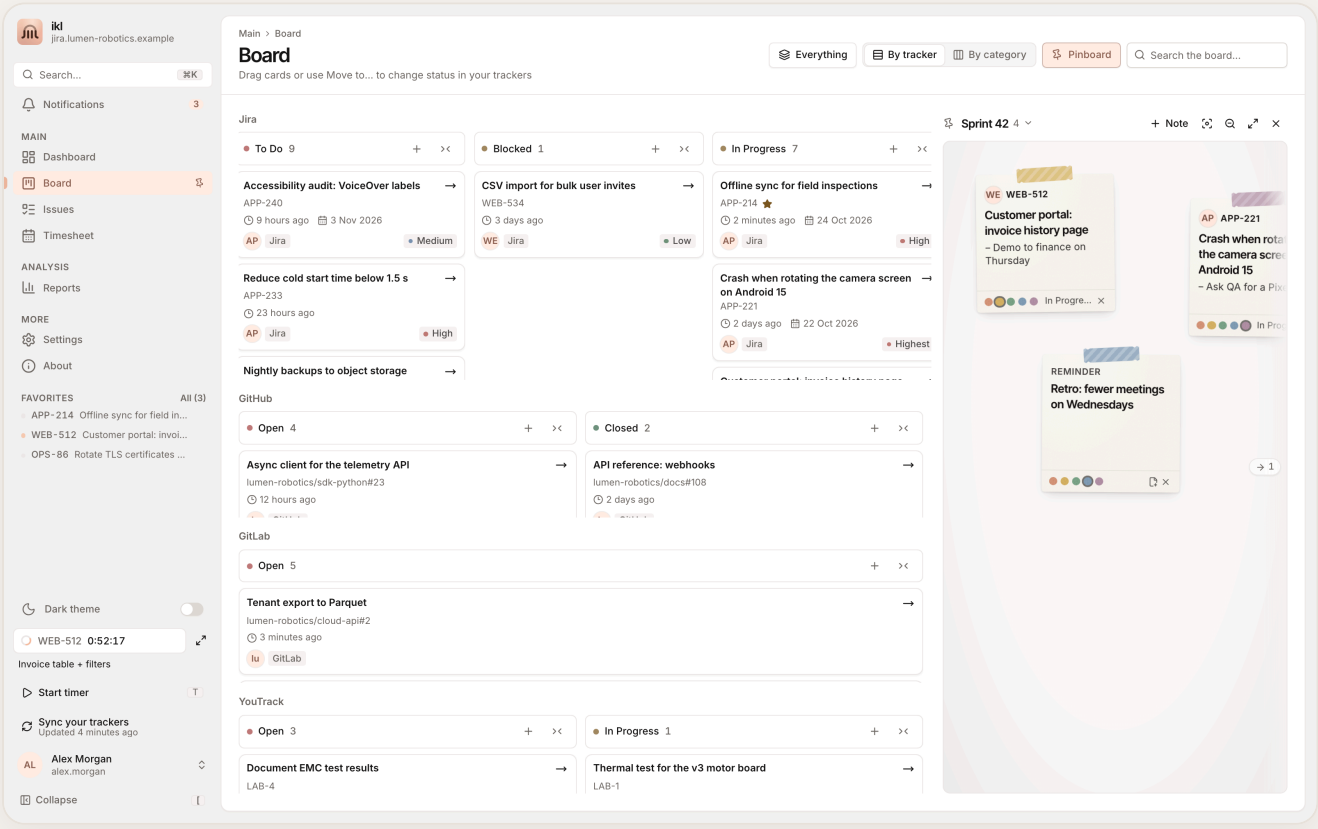
The system (or the browser) reports when you were away; inkledger asks whether to keep, remove or assign that time to another issue, so the timer never silently inflates.

PRODUCT TOUR · PLAN

Issues, search, detail and board.



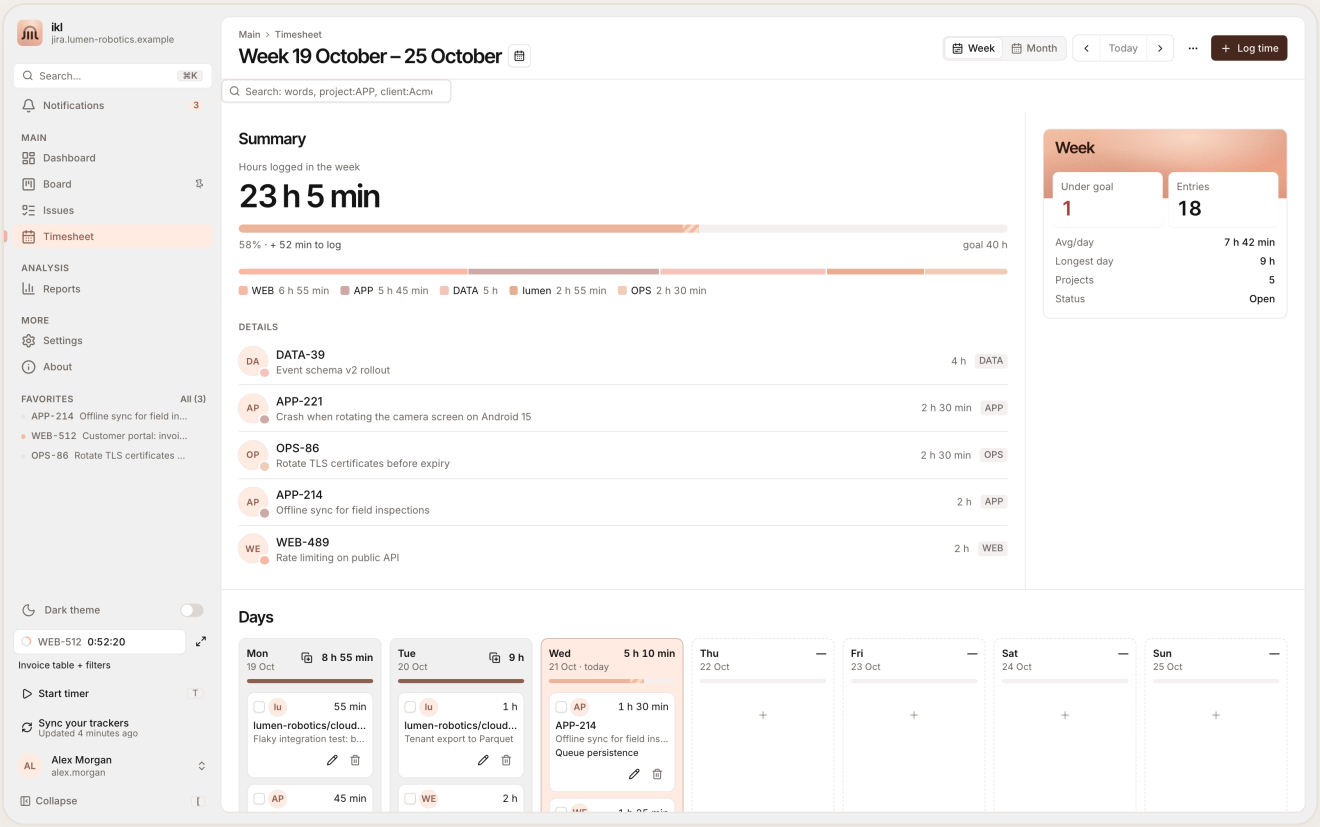
Issue detail: description, comments, history and transitions from the tracker, worklogs, logged versus estimate.



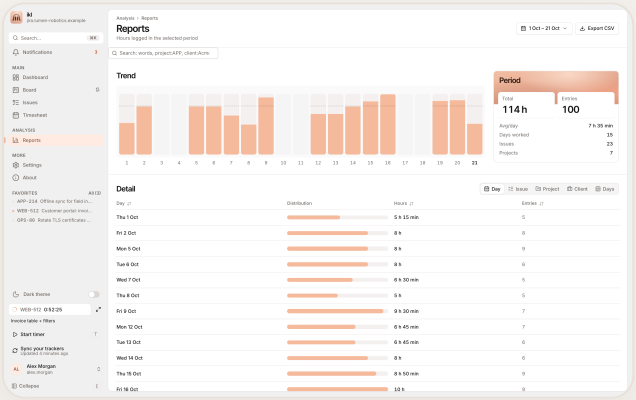
Board: dragging a card runs the tracker's transition; the pinboard keeps notes and issues next to it.

PRODUCT TOUR · REVIEW

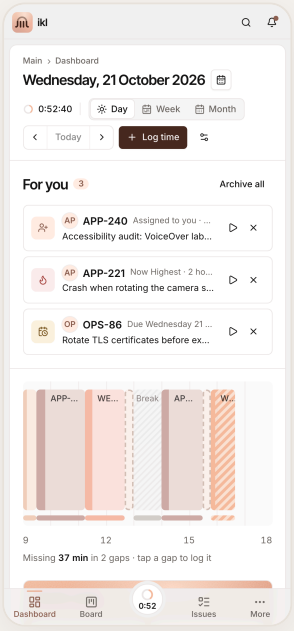
Weeks, months and reports.

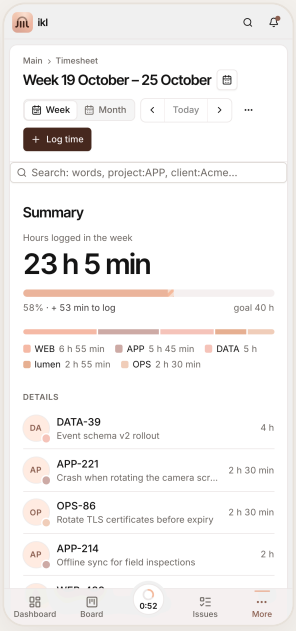


Timesheet: the week by project and by day, duplicate a day, bulk edits, close the week once delivered.



Reports: any range, grouped by day, issue or project, exported as CSV.

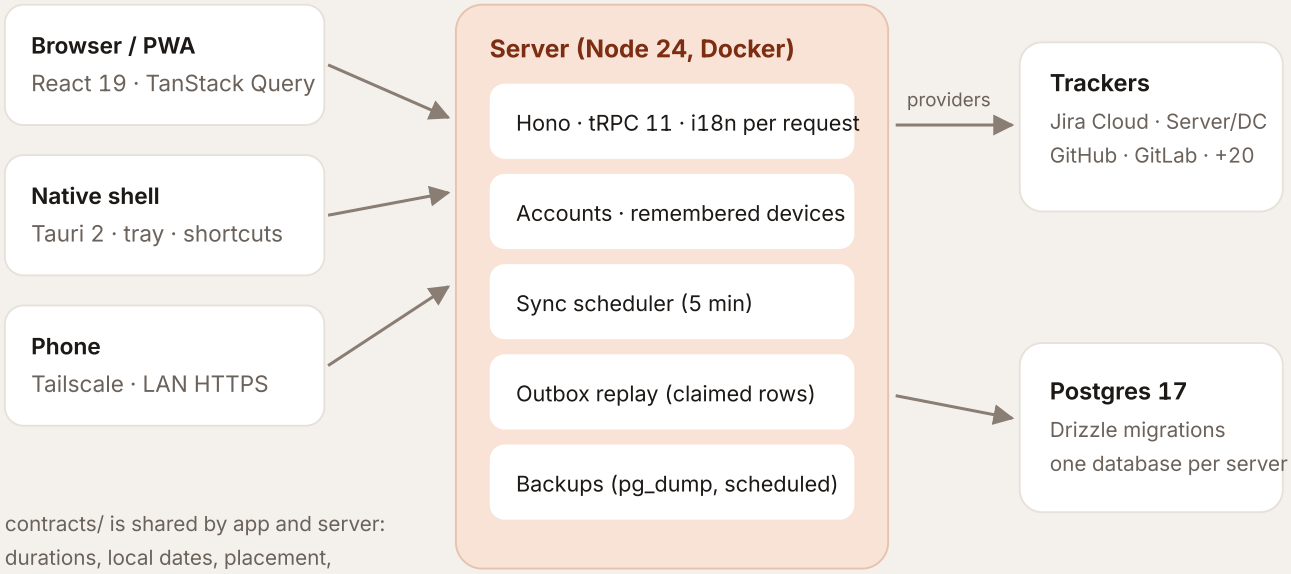




Everything is one keystroke away: `⌘K` jumps to any issue or action, `g` then a letter switches page, `T` starts a timer.

ARCHITECTURE

One server, many windows onto it.



contracts/ is shared by app and server:
durations, local dates, placement,
work schedules, dictionaries.

Thin clients, one source of truth

Every client is a window onto your own server, which every user of it shares. The native app is a Tauri shell that loads the server's UI, adding tray, global shortcuts, system idle and notifications. Phones reach the same server over Tailscale or the local Wi-Fi, so data never syncs between devices: there is only one copy.

One contract per tracker

Each tracker is an adapter behind one provider contract, checked by a shared conformance suite; a catalogue of 25 services records what each can do. Jira and GitHub are verified on real accounts. The others are built against HTTP simulators written from their documentation, traps included: they read at once and write only once the user allows it, until a real account verifies them.

Typed end to end

tRPC carries TypeScript types from the routers to the components; Drizzle types the queries; a test builds the database from the migrations and compares it with the schema the code uses, so drift is caught before a deploy.

KEY DECISIONS · DATA INTEGRITY

Time is never logged twice, and never lost.

OFFLINE OUTBOX

Queue, replay, adopt

A write that cannot reach the tracker is stored as an outbox entry plus a local placeholder worklog, in one transaction. Replays run oldest first and stop at the first network error. A request that timed out may have reached the tracker anyway, so before re-posting a create the replay looks for an identical worklog of the user (same start, same duration, created around the time of the request) and adopts it. That includes one a sync has already imported.

The development server and the container share one Postgres. Each entry is **claimed in the database** before it is sent, so two processes never post it twice. A claim expires if its process dies.

PLACEMENT

Overlaps are a choice, not an accident

Parallel work and AI agents make overlapping time real, so it is allowed. Each user sets a policy: warn or block. Every block is placed against what the day already holds: worklogs, timers, the working schedule and its breaks. The day timeline shows overlaps as lanes. Writes run under a per-user lock, so two tabs cannot both pass a check.

EDITS

Newest intent wins

An online edit drops any older queued edit of the same worklog, so a replay never rolls the tracker back. A delete that the tracker answers with 404 removes the local copy, and deleted worklogs wait 30 days in a trash. Being offline does not count toward the retry limit; only the tracker's refusals do.

TIME REGISTER

Where time lives

Trackers that keep time, such as Jira, get real worklogs, written through. Trackers without worklogs, such as GitHub, keep their time in inkledger's own register. On localized Jira, status categories are read by their language-independent key (`new` , `indeterminate` , `done`), never by name.

Signed in once, reachable everywhere, in your own scope.

SESSIONS

Accounts and remembered devices

The first start creates an admin who must change the password. The admin invites people with permissions (users, Store, backups, server), and each connects their own trackers. Signing out ends the session but the device stays remembered through a rotating, hashed token in its own HttpOnly cookie, so signing in again is one click. Tracker secrets never leave the server. If a tracker rejects a credential, that connection is marked "reconnect" and nobody is signed out.

CONNECTIONS

Several connections, several servers

A user can hold several connections per tracker type: test, rename, reconnect or archive them, and the time stays. Buckets narrow lists, board, palette and notifications on one device to chosen trackers or projects; personal totals stay global. The native app also keeps a list of servers. Cookies are scoped by port, so switching never signs another profile out.

DEVICES

Phones without a cloud

Tailscale publishes the server over HTTPS on the user's tailnet. Without it, the Mac creates a local certificate authority and serves the app on the Wi-Fi; a phone installs the certificate once from a QR code. Allowed host names are managed from the app, and a DNS-rebinding guard answers 421 to anything else.

LANGUAGES

One dictionary for app, server and menus, and one search language

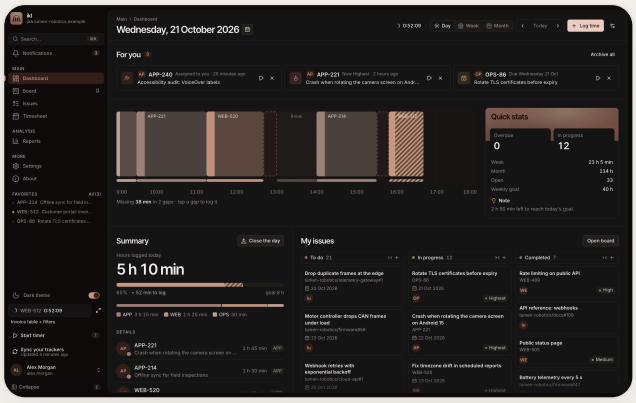
Texts live in shared dictionaries; the Italian one must have every English key or the typecheck fails. The browser sends its language with each request, so server errors match the screen, and the native menus follow the same choice. Search is one language over the whole catalogue, in English and Italian: words, keys and links (`APP-12` , `owner/repo#12`), and operators such as `project:` , `status:` , `updated:7d` , `@me` . What cannot be applied is shown, not dropped.

DESIGN

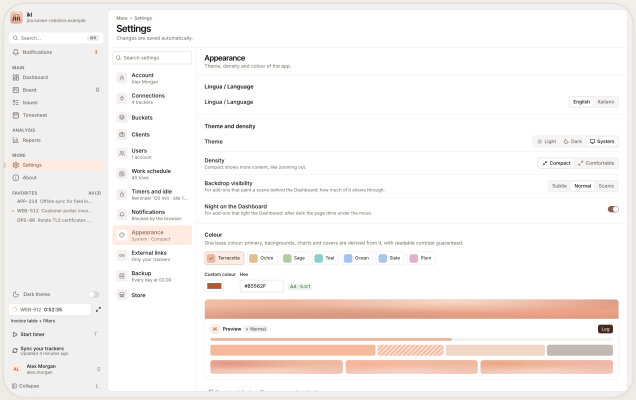
Warm, quiet and dense.

The interface follows warm neutrals. Colour is reserved for data: projects get soft, distinguishable tones, and charts and timelines carry the meaning. A theme colour picker changes the accent, and the in-app mark follows the theme gradient.

- **Design tokens only**: a lint step fails the build on raw colours, off-scale radii and default text sizes.
- **Compact by default**, "comfortable" on request; tabular numbers everywhere.
- **Accessible**: AA contrast, focus rings, screen reader labels, keyboard alternatives to drag and drop, reduced motion respected.
- **Consistent states**: every list has its loading, error and empty state; destructive actions ask with a red confirm.



The same tokens in dark mode.



Appearance: language, theme, density and theme colour.

QUALITY

Every bug becomes a test.

550+
unit and integration tests (Vitest), with Postgres in-process via PGlite

79
browser regressions (Playwright), each on its own throwaway database

6
gates before a deploy: lint, typecheck, tests, build, smoke, e2e

What is tested

- Durations and local dates across DST (Europe/Rome), placement and gaps, work schedules.
- The tRPC routers end to end against a real Postgres: sign-in hardening, remembered devices, sync, worklogs, the outbox with concurrent replays and adoption, week locks, reports, backups and restore. Every tracker adapter passes the same conformance suite.
- Schema drift between migrations and code, the en/it dictionaries, server messages per language.
- Browser flows that once broke: cutting gaps, drag and drop on the pinboard, profile switching, navigation memory, destructive confirms, the default language.

Safety in testing

Test servers run with `PROVIDERS_READ_ONLY=1`: every write to a tracker is refused at the HTTP layer. Unit tests have the network disabled unless a test stubs it.

Shipping

Production is a Docker Compose stack (app and Postgres). Migrations apply at start, scheduled `pg_dump` backups go to a folder outside the container, and a backup is taken before every deploy.

Audits

Before v1, a dedicated pass audited the server, the client logic and the UI for bugs and inconsistencies. It produced 59 findings; the confirmed ones were fixed before any new feature, from duplicated worklogs to stale data and missing confirmations.

OUTCOME

From a weekly chore to a daily habit.

What changed

- Time is logged while working: a timer per task and a glance at the timeline before leaving.
- The week closes itself: gaps are visible on the day they happen, so Friday is review, not archaeology.
- A tracker's downtime no longer means lost entries or duplicates.
- Work from every tracker (Jira, GitHub, GitLab, YouTrack…) lives in one timesheet, and time filed on Harvest, Toggl or Tempo follows by itself.
- The same server serves the desk, the laptop and the phone, with one copy of the data.

Next

- More languages through the shared dictionaries.
- Each tracker built on a simulator, verified on a real account.
- Team views for leads who approve timesheets.
- Calendar import to pre-fill meetings as worklogs.

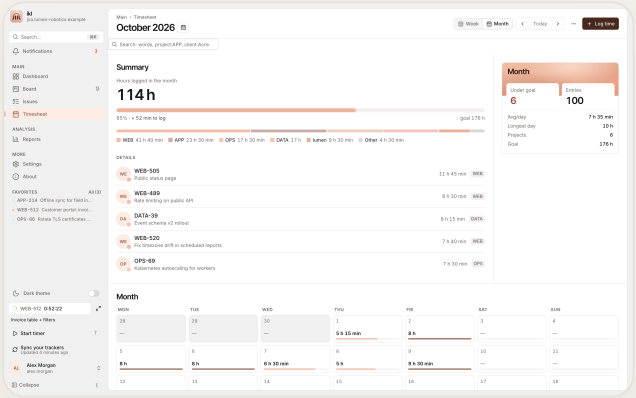
Try it

Clone the repository, create the secrets and start the stack:

```
docker compose up -d --build
```

Then open `http://127.0.0.1:47823`, sign in as admin, choose a new password and connect your trackers.

github.com/inkledger/inkledger
MIT licensed. Stars, issues and pull requests are welcome.



The month view: every day against its goal.